Aarogya360

Technical Requirements Document (TRD)

v2.0 – Revised After PRD Review and Gap Analysis

Aarogya360 Engineering Team

Version 2.0
June 20, 2026

Contents

1. Document Information

1.1 Purpose

This Technical Requirements Document (TRD) v2.0 supersedes v1.0. It incorporates the findings of the internal PRD Review (loophole analysis) and the Missing Requirements document, resolving all P0 items and addressing P1/P2 items either with a concrete decision or an explicitly flagged open item for future resolution.

1.2 What Changed From v1.0

- Multi-tenancy strategy explicitly decided (Row-Level Security)
- Technology stack confirmed and finalized, including mobile framework decision (React Native + Expo)
- Drug interaction detection redesigned as a Phase 2 rule-based engine, reconciling the AI Phase 5 contradiction
- Acceptance criteria added to all core functional requirements
- Appointment slot locking / concurrency strategy defined
- WhatsApp Business API added to the Communication Module
- DPDP Act 2023 and GST compliance sections added
- Razorpay confirmed as the payment gateway
- Subscription model defined: Free Trial followed by paid tiers
- Data model expanded with fields, relationships, indexes, and soft-delete strategy
- API versioning, FHIR R4 readiness, monitoring/alerting, and file storage specification added
- Minor patient and guardian consent model added
- Notification opt-out and communication audit logging added
- Staging environment and SLA measurement window explicitly defined

1.3 Related Documents

- Aarogya360 – Team Blueprint (v1.0)
- Aarogya360 – Product Requirements Document (v1.0)
- Aarogya360 – Missing Requirements & Improvements (Gap Analysis)

- Aarogya360 – PRD Review (Loophole Analysis)

1.4 Version History

Version	Date	Author	Summary
1.0	–	Engineering Team	Initial TRD derived from PRD v1.0
2.0	June 20, 2026	Engineering Team	Full revision addressing PRD review loopholes and missing requirements

2. System Overview

2.1 Architectural Style

Aarogya360 is built as a **Modular Monolith**: a single deployable backend service internally organized into strictly bounded modules, each owning its own router, service, repository, models, and schemas. This is the right choice at current scale – faster development, simpler operations, lower cost – while preserving clean boundaries for a future service extraction if warranted.

2.2 High-Level Components

- **Clinic Dashboard** – Next.js 14 web application for clinic owners, doctors, receptionists, lab staff
- **Patient Mobile App** – React Native + Expo application, the patient's lifelong health passport
- **Backend API** – FastAPI modular monolith exposing versioned REST APIs
- **Rule-Based Drug Interaction Engine** – deterministic lookup service, shipped in Phase 2
- **AI Layer** – OCR, summarization, RAG-based assistant, and LLM-assisted explanatory drug/lab analysis, shipped in Phase 5
- **Communication Layer** – Email, SMS, WhatsApp Business API, Push Notifications
- **ABHA / FHIR Integration Layer** – adapter module, Phase 6, designed against FHIR R4 from Phase 1 data modeling onward
- **Data Stores** – PostgreSQL (primary, row-level multi-tenant), Redis (cache/broker), Vector DB (embeddings), AWS S3 (files)

2.3 Module List

Authentication, Multi-Tenancy, Clinics, Doctors, Patients, Appointments, Consultations, Prescriptions, Drug Interaction (rule engine), Laboratory, Billing, Communication/Notifications, Analytics, Follow-Up, AI Services, ABHA/FHIR Integration.

3. Multi-Tenancy Strategy (Resolved)

3.1 Decision

Row-Level Security with a shared database and a `clinic_id` column on every tenant-scoped table.

3.2 Rationale

Approach	Pros	Cons
Database-per-tenant	Strongest isolation	Very high operational cost; impractical at hundreds of clinics
Schema-per-tenant	Good isolation	Migration complexity grows linearly with tenant count
Row-Level Security (chosen)	Lowest cost, simplest operations, scales to thousands of tenants	Requires strict query discipline; isolation enforced in code, not by the DB engine alone

3.3 Implementation Rules

1. Every tenant-scoped table includes a non-nullable `clinic_id` foreign key.
2. A shared `BaseRepository` class injects `clinic_id` into every query automatically; no repository method may accept a raw, unscoped query.
3. PostgreSQL native Row-Level Security (RLS) policies are enabled as a second enforcement layer, using a session variable set to the authenticated `clinic_id` at the start of each request.
4. Cross-tenant patient data visibility is only possible via an explicit `consent_links` record (see Chapter 6).

Acceptance Criteria: 100% of automated integration tests include at least one negative test asserting that a request scoped to Clinic A cannot retrieve, modify, or list any row belonging to Clinic B, including via Patient/AI endpoints.

4. Technology Stack (Confirmed)

4.1 Backend

Python 3.11+, FastAPI (async), SQLAlchemy (async ORM), PostgreSQL 15+, Redis (cache + Celery broker), Celery (background jobs).

4.2 Frontend Dashboard

Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, Zustand, TanStack Query.

4.3 Mobile App

React Native + Expo + Expo Router – confirmed. Shares TypeScript/React patterns with the dashboard team, reducing context-switching and easing hiring.

4.4 AI Layer

OpenAI GPT models, Embeddings API, RAG architecture, OCR processing pipeline (Phase 5). Drug interaction detection is split: a deterministic rule engine (Phase 2) plus an LLM-based explanatory layer (Phase 5) – see Chapter 9.

4.5 Payments

Razorpay – confirmed as the UPI/card payment gateway. Supports India-specific settlement, webhook-based payment confirmation, and refund handling. A parallel **cash payment record flow** is required in the Billing module for clinics that accept cash (see Chapter 10).

4.6 Communication

SMS gateway (e.g., MSG91/Twilio – to be finalized at implementation), Email (transactional provider e.g. SES/SendGrid), **WhatsApp Business API** (via an official BSP such as Gupshup or Interakt), Push Notifications (FCM/APNs via Expo push service).

4.7 Infrastructure

Docker, AWS ECS, AWS RDS (PostgreSQL), AWS ElastiCache (Redis), AWS S3, GitHub Actions, Terraform.

4.8 Drug Interaction Data Source

[OPEN ITEM] Not yet finalized. Candidate options under evaluation: (a) RxNorm combined with a manually curated India brand-name-to-generic mapping table, or (b) a licensed Indian drug database from a commercial vendor. This decision must be closed before Phase 2 development of the Prescription module begins, as it directly affects the rule engine's schema. Tracked as a blocking pre-Phase-2 task.

5. System Architecture

5.1 Module Internal Structure

Router (HTTP layer) → Service (business logic) → Repository (data access) → Models (ORM) → Schemas (Pydantic validation). All five layers are mandatory per module; no module may skip the service layer and call the repository directly from a router.

5.2 Inter-Module Communication

Modules call each other only through public service interfaces, never through direct cross-module database access. Example: the Prescription module calls `PatientService.get_current_medications()` rather than querying the patient schema directly.

5.3 Background Processing

Celery + Redis handle: prescription OCR, AI health summaries, lab report analysis, email/SMS/WhatsApp dispatch, follow-up reminders, and recall campaigns. All background jobs are idempotent and retried with exponential backoff on failure.

5.4 Environments

Development, Staging, and Production are mandatory and isolated – separate databases, separate S3 buckets, separate API keys for all third-party integrations (Razorpay test mode in staging, WhatsApp sandbox in staging). No testing is permitted against production tenant data under any circumstance.

Acceptance Criteria: No CI/CD pipeline may deploy directly to production without first deploying to staging and passing the automated smoke-test suite.

6. Data Model

6.1 Soft Delete and Audit Strategy

All clinical and financial entities use soft deletion (`deleted_at` timestamp, nullable) rather than hard deletes, to preserve the legal and clinical audit trail. Every mutation on patient-identifiable data is mirrored into `audit_logs`.

6.2 Core Schema

```
clinics(
  id UUID PK, name, address, gstin, specialty,
  subscription_plan_id FK, trial_ends_at,
  created_at, deleted_at NULL
)

users(
  id UUID PK, clinic_id FK INDEX, role ENUM,
  name, email UNIQUE, phone, password_hash,
  mfa_enabled BOOL, created_at, deleted_at NULL
)

patients(
  id UUID PK, name, dob, gender, phone INDEX,
  abha_id NULLABLE, qr_code UNIQUE,
  is_minor BOOL, guardian_patient_id FK NULLABLE,
  created_at, deleted_at NULL
)

clinic_patients(
  clinic_id FK, patient_id FK, consent_status ENUM,
  linked_at, revoked_at NULL,
  PRIMARY KEY(clinic_id, patient_id)
)

doctors(
  id UUID PK, user_id FK UNIQUE, specialty,
  qualification, consultation_fee DECIMAL
)

doctor_availability(
  id UUID PK, doctor_id FK INDEX, day_of_week,
  start_time, end_time, slot_duration_minutes
)

appointments(
```

```
  id UUID PK, clinic_id FK INDEX, doctor_id FK INDEX,
  patient_id FK INDEX, slot_start TIMESTAMP,
  slot_end TIMESTAMP, status ENUM, token_number,
  UNIQUE(doctor_id, slot_start) -- enforces slot locking
)

consultations(
  id UUID PK, appointment_id FK UNIQUE, symptoms TEXT,
  examination TEXT, diagnosis TEXT, treatment_plan TEXT,
  created_at
)

prescriptions(
  id UUID PK, consultation_id FK UNIQUE, created_at
)

prescription_items(
  id UUID PK, prescription_id FK INDEX, medicine_name,
  dosage, frequency, duration,
  interaction_warning_ack BOOL DEFAULT FALSE
)

drug_interaction_rules(
  id UUID PK, drug_a, drug_b, severity ENUM,
  description, source_reference
)

allergies(
  id UUID PK, patient_id FK INDEX, allergen,
  reaction_severity ENUM, recorded_at
)

lab_orders(
  id UUID PK, consultation_id FK INDEX, test_name,
  status ENUM, created_at
)

lab_reports(
  id UUID PK, lab_order_id FK UNIQUE, file_url,
  structured_data JSONB, abnormal_flags JSONB,
  uploaded_at
)

invoices(
  id UUID PK, clinic_id FK INDEX, patient_id FK INDEX,
  amount DECIMAL, gst_amount DECIMAL, hsn_code,
  payment_method ENUM, razorpay_payment_id NULLABLE,
  status ENUM, created_at, deleted_at NULL
)

consent_links(
  id UUID PK, patient_id FK INDEX, source_clinic_id FK,
  target_clinic_id FK, status ENUM,
  granted_at, revoked_at NULL
)

guardian_consents(
  id UUID PK, minor_patient_id FK INDEX,
```

```
  guardian_user_id FK, consent_type ENUM,
  granted_at, revoked_at NULL
)

subscription_plans(
  id UUID PK, name, max_doctors INT, max_patients INT,
  price_monthly DECIMAL, features JSONB
)

notification_preferences(
  id UUID PK, patient_id FK UNIQUE, email_opt_in BOOL,
  sms_opt_in BOOL, whatsapp_opt_in BOOL,
  marketing_opt_in BOOL DEFAULT FALSE
)

communication_logs(
  id UUID PK, patient_id FK INDEX, channel ENUM,
  template_id FK, status ENUM, sent_at, delivered_at NULL
)

audit_logs(
  id UUID PK, actor_user_id FK INDEX, action,
  entity_type, entity_id, clinic_id FK INDEX, timestamp
)
```

6.3 Key Indexes

- `appointments(doctor_id, slot_start)` – UNIQUE, prevents double-booking at the database level
- `patients(phone)`, `patients(qr_code)` – fast lookup for search and check-in
- `users(clinic_id)`, all tenant-scoped tables on `clinic_id` – supports RLS query performance
- `audit_logs(clinic_id, timestamp)` – supports compliance reporting queries

6.4 FHIR R4 Readiness

Core clinical entities (`consultations`, `prescriptions`, `lab_reports`, `patients`) are designed so that each maps cleanly to a corresponding FHIR R4 resource (`Encounter`, `MedicationRequest`, `DiagnosticReport`, `Patient`) without structural rework. A FHIR mapping layer will be implemented in Phase 6, not retrofitted onto the relational schema.

7. Appointment Concurrency & Slot Locking

7.1 Problem

Two receptionists, or a receptionist and the patient app, may attempt to book the same doctor slot simultaneously.

7.2 Solution

1. A UNIQUE database constraint on `(doctor_id, slot_start)` guarantees only one appointment can ever be persisted for a given slot, regardless of application-layer race conditions.
2. The booking service wraps the insert in a database transaction; on a unique-constraint violation, the API returns `409 Conflict` with an error code `SLOT_ALREADY_BOOKED`.
3. A short-lived Redis lock (`lock:doctor:{id}:{slot}`, TTL 10s) is acquired before presenting slot availability, to reduce – though not solely rely on – race conditions at the UI layer.
4. All booking requests include a client-generated idempotency key to prevent duplicate submissions from network retries.

Acceptance Criteria: Under a concurrency test of 50 simultaneous booking requests for the same slot, exactly one request succeeds with `201 Created` and all others receive `409 Conflict`; zero duplicate appointment rows are created.

8. Drug Interaction & AI Phase Reconciliation

8.1 Problem Being Resolved

The original PRD listed drug warnings under the Prescription module (Phase 2) while placing all AI services, including drug interaction detection, in Phase 5 – a direct contradiction.

8.2 Resolved Design

- **Phase 2 – Rule-Based Engine (mandatory, ships with Prescriptions):** A deterministic lookup against `drug_interaction_rules` checks every new prescription item against the patient's active medication list. Any match blocks silent submission and requires explicit doctor acknowledgement (`interaction_warning_ack`).
- **Phase 5 – LLM Explanatory Layer (additive, not a replacement):** An LLM call may be used only to generate a plain-language explanation of an already-flagged interaction. It never independently flags a new interaction and never overrides the rule engine.

Acceptance Criteria: A prescription containing two medicines present together in `drug_interaction_rules` cannot be saved with status "finalized" unless `interaction_warning_ack = TRUE` on the relevant line item; this is enforced at the API layer, not only in the UI.

8.3 Allergy Cross-Check

The same Phase 2 rule engine also checks new prescription items against the patient's recorded `allergies` table and raises a blocking warning on a match, using the same acknowledgement mechanism.

9. AI Subsystem Design (Phase 5)

9.1 Architectural Guardrail

No AI endpoint may write directly to a patient record. Every AI endpoint returns a suggestion object; a separate, human-triggered API call commits any resulting change. This is enforced by giving AI-layer service accounts no write permissions on clinical tables at the database role level, not only at the application layer.

9.2 Prescription OCR Pipeline

Image uploaded via pre-signed S3 URL → Celery OCR task → structured extraction (medicine, dosage, frequency) → doctor reviews and confirms before the record is saved as official.

9.3 Health Summary Generation

Aggregates recent consultations, active prescriptions, and chronic condition flags into a structured prompt; output cached per patient, invalidated on new clinical data.

9.4 Lab Report Analysis

Structured values are compared against reference ranges programmatically to flag abnormalities; the LLM is used only to generate a plain-language explanation of an already-flagged abnormality.

9.5 AI Health Assistant – RAG Pipeline

Patient records are chunked, embedded, and stored in a vector database strictly namespaced by `patient_id`. Retrieval is scoped exclusively to the authenticated patient; the LLM is instructed to answer only from retrieved context and decline otherwise. Every query/response pair is logged.

Acceptance Criteria: A penetration test attempting to retrieve Patient B's embedded records while authenticated as Patient A returns zero matching vector results.

10. Billing, GST, and Payments

10.1 GST-Compliant Invoicing

Every invoice includes GSTIN (clinic's), HSN/SAC code per line item, and a tax breakdown (CGST/SGST or IGST as applicable). Invoice numbering follows a sequential, non-reusable format per clinic per financial year, as required for GST audit purposes.

10.2 Payment Methods

- **Razorpay** – UPI, card, and netbanking payments, with webhook-based payment status confirmation
- **Cash** – a manual cash-received record flow, logged against the same invoice entity with `payment_method = CASH`, requiring receptionist confirmation

Acceptance Criteria: An invoice cannot transition to status `PAID` for an online payment without a verified Razorpay webhook signature matching the invoice's `razorpay_payment_id`.

10.3 Subscription Model

Free Trial followed by paid tiers.

Plan	Duration	Max Doctors	Max Patients	Notes
Free Trial	14 days	2	200	Full feature access; auto-downgrades to read-only on expiry unless upgraded
Basic	Monthly	3	1,000	Core clinical workflow, no AI features
Pro	Monthly	10	10,000	Core workflow + AI services + WhatsApp
Enterprise	Monthly	Unlimited	Unlimited	All features, priority support, custom SLAs

[OPEN ITEM] Exact pricing (INR/month per tier) is to be finalized with business stakeholders before public launch; the tier structure and feature gating logic above are final for engineering purposes.

11. Communication & Notification Module

11.1 Channels

Email, SMS, **WhatsApp Business API**, and Push Notifications. WhatsApp is treated as a first-class channel, not an afterthought, given its dominant reach in the target market.

11.2 Use Cases by Channel

Channel	Use Cases
Email	Appointment confirmations, prescriptions (PDF), invoices, lab reports
SMS	OTPs, appointment reminders, queue status updates
WhatsApp	Appointment reminders, prescription delivery, invoice delivery, lab report delivery
Push	Real-time in-app alerts for appointments, prescriptions, reports

11.3 Template Management

All outbound messages are rendered from versioned templates stored per channel, supporting variable interpolation (patient name, appointment time, etc.) and multi-language variants.

11.4 Notification Preferences and Opt-Out

Every patient has a `notification_preferences` record. Marketing/recall communications require `marketing_opt_in = TRUE`; transactional messages (OTP, appointment confirmation) are sent regardless of marketing opt-out, per standard practice, but channel-level opt-out (e.g., disabling WhatsApp specifically) is honored even for transactional messages where a fallback channel exists.

11.5 Delivery Tracking and Audit

Every message is recorded in `communication_logs` with channel, template, status (queued/sent/delivered/failed), and timestamps, forming the communication audit trail required for compliance review.

Acceptance Criteria: A bulk recall campaign targeting 500 patients with `marketing_opt_in = FALSE` sends to zero of those patients and logs a skipped status for each.

12. Security, Privacy, and Compliance

12.1 DPDP Act 2023 Compliance

- Explicit, granular consent captured at patient registration for data processing purposes
- Right to access: patients can export their full health record via the mobile app
- Right to erasure: supported via soft-delete plus a documented hard-delete procedure for closed accounts, subject to applicable medical record retention law
- Data Protection Officer (DPO) contact and grievance redressal process to be documented operationally outside this TRD
- Breach notification procedure documented in the incident response runbook

12.2 Minor Patient and Guardian Consent

- Patients flagged `is_minor = TRUE` have a mandatory `guardian_patient_id` link
- All consent actions (registration, cross-clinic linking, AI assistant usage) for a minor require a corresponding `guardian_consents` record signed by the guardian's authenticated account
- Sensitive record categories (to be defined with clinical/legal input) may restrict guardian visibility for minors above a configurable age threshold, per applicable Indian medical privacy norms

[OPEN ITEM] The exact age threshold and sensitive-category list for restricted guardian visibility requires clinical and legal sign-off before Phase 1 patient registration ships.

12.3 Authentication & Authorization

JWT access tokens (short-lived) + rotated refresh tokens; RBAC enforced at the service layer for every endpoint; optional OTP-based MFA for clinic owners and doctors.

12.4 Data Protection

Encryption at rest for database and S3; TLS/HTTPS enforced everywhere; column-level encryption for ABHA ID and other sensitive identifiers.

12.5 File Storage Specification

- Provider: AWS S3, private buckets, pre-signed time-limited upload/download URLs

- Supported formats: PDF, JPEG, PNG for prescriptions/reports; max file size 10MB per file
- Retention: clinical documents retained for a minimum of the legally mandated medical record period (to be confirmed per applicable state/central regulation, default working assumption 8 years); structured data retained indefinitely subject to DPDP erasure requests
- Virus/malware scanning on upload prior to persistence

12.6 Audit Logging

Every create/read/update/delete on patient-identifiable data is recorded in `audit_logs`, immutable, retained per the data retention policy above.

12.7 Rate Limiting

Per-IP and per-user rate limiting at the API gateway layer, with stricter limits on authentication and AI endpoints.

13. API Requirements

13.1 Versioning Strategy

All endpoints are served under `/api/v1/`. Breaking changes are introduced under a new version prefix (`/api/v2/`); `v1` is maintained in parallel for a minimum deprecation window of 6 months after `v2` ships, to avoid breaking unupdated mobile app installs.

13.2 Conventions

JSON over HTTPS; Bearer JWT auth; standard error envelope {`error_code, message, details`}; pagination via `limit`/`offset`; all list endpoints support filtering by `clinic_id` scope implicitly via the authenticated session.

13.3 Representative Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate, issue tokens
POST	/api/v1/appointments	Book appointment (slot-locked)
POST	/api/v1/prescriptions	Create prescription (rule-engine checked)
POST	/api/v1/invoices	Create GST-compliant invoice
POST	/api/v1/payments/razorpay/webhook	Razorpay payment confirmation
POST	/api/v1/notifications/whatsapp/send	Dispatch WhatsApp message from template
POST	/api/v1/ai/assistant/query	Patient AI assistant query (RAG-scoped)
GET	/api/v1/patients/{id}/export	DPDP data export request

Acceptance Criteria: Every endpoint listed in this chapter has an OpenAPI schema generated automatically by FastAPI, kept in sync with implementation, and used directly as the contract for QA test generation.

14. Monitoring, Logging, and Reliability

14.1 Monitoring Stack

Application performance monitoring and error tracking (e.g., Sentry) integrated at the FastAPI middleware level; infrastructure metrics via AWS CloudWatch; centralized structured logging (JSON logs) shipped to a log aggregation service.

14.2 Alerting

- Error rate spike alerts (e.g., >2% 5xx rate over 5 minutes)
- Latency degradation alerts (P95 >2s sustained)
- Celery queue backlog growth alerts
- Razorpay webhook failure alerts

14.3 On-Call & Incident Response

A documented on-call rotation and incident response runbook covering detection, triage, communication, and post-incident review, to be maintained outside this TRD but referenced from it.

14.4 SLA Definition

99.5% uptime, measured monthly, on core clinical workflow endpoints only (auth, appointments, consultations, prescriptions). Planned maintenance windows, announced at least 48 hours in advance and scheduled outside clinic operating hours (default 11 PM–3 AM IST), are excluded from the SLA calculation. This equates to a maximum of approximately 3 hours 39 minutes of unplanned downtime per month.

14.5 Backup & Disaster Recovery

Automated daily RDS snapshots with point-in-time recovery; S3 versioning enabled; documented recovery runbook with target RTO of 4 hours and RPO of 1 hour for the primary database.

15. Non-Functional Requirements (With Acceptance Criteria)

15.1 Performance

Acceptance Criteria: P95 API latency under 500ms for standard CRUD operations, and under 2s for AI-backed endpoints, measured under a simulated load of 500 concurrent users across 50 tenants.

15.2 Scalability

Acceptance Criteria: The system supports linear horizontal scaling of API instances behind a load balancer with no shared in-memory state; load tests confirm throughput scales by at least 1.7x when API instance count is doubled.

15.3 Availability

Acceptance Criteria: Per the SLA definition in Chapter 13.4.

15.4 Search Performance

Acceptance Criteria: Patient search by name, phone, or ID returns results in under 1 second at P95 for a clinic database of up to 50,000 patients.

15.5 Maintainability

Acceptance Criteria: Minimum 80% unit test coverage on service-layer business logic per module, enforced as a CI pipeline gate that blocks merge below threshold.

16. Engineering Roadmap Alignment (Reconciled)

16.1 Phase 1 – Core Foundation

Authentication, Multi-Tenancy (RLS), Clinic, Doctor, Patient (including minor/guardian model), Appointment module with slot locking. FHIR-aware schema design begins here.

16.2 Phase 2 – Clinical Workflow

Consultation, Prescription, **rule-based Drug Interaction Engine**, Allergy Management, Laboratory, and GST-compliant Billing with Razorpay + cash flow.

16.3 Phase 3 – Subscriptions, Billing Depth, and Analytics

Free Trial and tiered subscription enforcement, revenue reporting, Clinic/Doctor/Patient KPI dashboards.

16.4 Phase 4 – Patient Mobile App

React Native + Expo app: Health Passport, QR identity, records timeline, notification preferences. Sequenced after Phase 1–3 are battle-tested, so the Health Passport is built on clean clinic-side data.

16.5 Phase 5 – AI Services

OCR, health summaries, LLM-explanatory drug/lab analysis layered onto the Phase 2 rule engine, and the RAG-based AI Health Assistant.

16.6 Phase 6 – ABHA / FHIR Integration

ABHA ID linking, ABDM consent-manager callback handling, FHIR R4 resource mapping for Consultation, Prescription, and LabReport entities.